

Operating System Fundamentals

What is an Operating System?

An operating system (OS) is system software that manages computer hardware and software resources while providing common services to programs. It sits between the user and the hardware, acting as a resource manager and an extended machine. Without an OS, every application would need to communicate directly with hardware, making software development extremely difficult.

Core Responsibilities of an OS

Process management involves creating, scheduling, and terminating processes to ensure the CPU is utilized efficiently. Memory management allocates and reclaims RAM so multiple programs can run simultaneously without conflict. File system management organizes data on disks into files and directories. Device management provides a uniform interface to hardware peripherals through device drivers. Security and access control protect system resources from unauthorized use.

Types of Operating Systems

A Batch OS collects jobs and runs them without user interaction, ideal for repetitive tasks. A Time-Sharing OS allows multiple users to share the CPU by rapidly switching between processes, giving the illusion of simultaneous execution. A Real-Time OS guarantees responses within strict time deadlines, used in embedded systems like pacemakers and aircraft controls. A Distributed OS manages a group of independent computers so they appear as a single system. A Mobile OS, such as Android or iOS, is optimized for battery life and touch interfaces.

Kernel and User Space

The kernel is the core of the OS that runs with full hardware privileges. It handles the most critical tasks such as process scheduling and memory allocation. User space is where applications run with restricted privileges; they request kernel services through system calls. This separation protects the OS from faulty or malicious programs. A monolithic kernel includes all OS services in one large binary, while a microkernel keeps only the bare minimum in kernel space.

Process Management & Scheduling

Process vs Thread

A process is an independent program in execution. It has its own address space, registers, heap, and stack. Creating a process is expensive because the OS must allocate separate memory. A thread is a lightweight unit of execution within a process. Threads share the same memory space but have separate stacks and registers. Multi-threading allows a single application to perform multiple tasks concurrently, improving responsiveness and throughput.

Process States

Every process passes through several states during its lifetime. In the New state, the process is being created. In the Ready state, it is waiting for CPU time. In the Running state, the CPU is actively executing its instructions. In the Waiting state, it is blocked waiting for an I/O event or signal. In the Terminated state, execution is complete. The Process Control Block (PCB) stores all information about a process including its state, program counter, and memory limits.

CPU Scheduling Algorithms

First-Come First-Served (FCFS) processes jobs in arrival order. It is simple but can lead to long waiting times if a slow job arrives first (convoy effect). Shortest Job First (SJF) picks the process with the smallest burst time next, minimizing average waiting time but requiring knowledge of future burst times. Round Robin assigns a fixed time slice to each process in circular order, ensuring fairness. Priority Scheduling runs the highest-priority process first; lower-priority processes may starve. Multi-Level Feedback Queue dynamically adjusts a process's priority based on its behavior.

Deadlock

A deadlock is a situation where a set of processes are permanently blocked because each is waiting for a resource held by another. The four necessary Coffman conditions are mutual exclusion, hold and wait, no preemption, and circular wait. Deadlock prevention breaks one of these conditions by design. Deadlock avoidance uses algorithms like Banker's Algorithm to allocate resources safely. Deadlock detection allows deadlocks to occur but recovers by killing or rolling back processes.

Memory Management

Memory Hierarchy

Memory is organized in a hierarchy based on speed, size, and cost. Registers sit at the top and are the fastest but hold only a few bytes. Cache memory (L1, L2, L3) is slightly larger and slower but still very fast. Main memory (RAM) is accessible in nanoseconds and stores running programs. Secondary storage such as SSDs and hard drives is slow but persistent and large. The OS manages this hierarchy to keep frequently used data as close to the CPU as possible.

Paging

Paging divides physical memory into fixed-size blocks called frames and logical memory into equal-size blocks called pages. The OS maintains a page table for each process that maps logical page numbers to physical frame numbers. Paging eliminates external fragmentation because any free frame can be assigned to any page. However, it can cause internal fragmentation when a process does not fully use its last page.

Segmentation

Segmentation divides a program's memory into variable-size logical units called segments, each representing a meaningful part such as code, data, or stack. Unlike paging, segment sizes match the programmer's view of memory. Segmentation supports protection and sharing at a logical level but can cause external fragmentation over time as segments are loaded and unloaded from memory.

Virtual Memory

Virtual memory allows a process to use more memory than physically available by storing inactive pages on disk and loading them on demand. This technique, called demand paging, triggers a page fault when a required page is not in RAM. The OS then swaps a page out and loads the needed page in. Page replacement algorithms such as LRU (Least Recently Used), FIFO, and Optimal decide which page to evict. Thrashing occurs when excessive page faults slow the system down.

File Systems & I/O Management

File System Concepts

A file system is the method an OS uses to organize and retrieve data on a storage device. It manages directories, metadata such as file size and timestamps, permissions, and physical block allocation. Common file systems include FAT32 and NTFS on Windows, ext4 on Linux, and APFS on macOS. Each has different strengths in terms of file size limits, journaling support, and performance.

Directory Structure

Directories organize files in a hierarchical tree structure. A single-level directory puts all files in one folder, which causes naming conflicts. A two-level directory gives each user a separate folder. A tree-structured directory is the most common approach, where paths like /home/user/documents navigate through nested folders. An acyclic-graph directory allows files to be shared via links without duplication.

Disk Scheduling

The disk head must physically move to read or write data on a spinning disk. Disk scheduling algorithms minimize this movement. FCFS serves requests in order of arrival but can result in high seek times. SSTF (Shortest Seek Time First) moves the head to the nearest request, reducing movement but potentially starving far requests. The SCAN algorithm moves the head in one direction serving requests until it reaches the end, then reverses, behaving like an elevator.

I/O Management

The OS provides a uniform interface to I/O devices through device drivers. Programmed I/O makes the CPU poll the device status continuously, wasting CPU cycles. Interrupt-Driven I/O lets the device notify the CPU via an interrupt when it is ready, freeing the CPU for other tasks. Direct Memory Access (DMA) allows the device to transfer data directly to memory without CPU involvement, maximizing throughput for large data transfers such as disk reads.

Introduction to Data Structures

What is a Data Structure?

A data structure is a way of organizing, storing, and managing data so it can be accessed and modified efficiently. The choice of data structure directly affects the performance of an algorithm. For example, finding an element is $O(1)$ in a hash table but $O(n)$ in a linked list. Understanding trade-offs between time and space complexity is essential for writing efficient programs.

Linear vs Non-Linear Structures

Linear data structures store elements in a sequential order where each element has a unique predecessor and successor. Examples include arrays, linked lists, stacks, and queues. Non-linear data structures store elements in a hierarchical or network arrangement. Examples include trees and graphs. Choosing between linear and non-linear structures depends on the relationships between data elements.

Big-O Notation

Big-O notation describes the worst-case time or space complexity of an algorithm as the input size n grows toward infinity. $O(1)$ means constant time regardless of input size. $O(\log n)$ represents logarithmic growth, seen in binary search. $O(n)$ is linear, typical of a simple loop. $O(n \log n)$ is seen in efficient sorting algorithms. $O(n^2)$ occurs in nested loops. Understanding Big-O allows developers to compare algorithms and predict performance at scale.

Abstract Data Types

An Abstract Data Type (ADT) defines a data structure by its behavior rather than its implementation. It specifies what operations are supported and their expected results without revealing how they work internally. For example, a Stack ADT defines push, pop, and peek operations. The underlying implementation could use an array or a linked list. This separation of interface from implementation promotes modularity and code reuse.

Arrays & Linked Lists

Arrays

An array stores elements in contiguous memory locations. Each element is accessed in constant time $O(1)$ using its index. Arrays are the simplest and most widely used data structure. They are efficient for random access and iterating over elements. However, insertion and deletion in the middle of an array require shifting elements, making these operations $O(n)$. Arrays have a fixed size in most languages, though dynamic arrays like Python lists resize automatically.

Linked Lists

A linked list consists of nodes where each node stores a value and a pointer to the next node. Memory is non-contiguous, so insertion and deletion at a known position are $O(1)$. However, accessing an element by index requires traversing from the head, making it $O(n)$. Linked lists are preferred when frequent insertions and deletions are needed and random access is not required.

Singly, Doubly, and Circular Linked Lists

In a singly linked list, each node points only to the next node, allowing traversal in one direction. In a doubly linked list, each node has both a next and a previous pointer, enabling traversal in both directions and making deletion easier when the node reference is available. A circular linked list connects the last node back to the first, useful for applications like round-robin scheduling and media playlists.

When to Use Array vs Linked List

Use an array when you need fast random access by index, elements are of fixed count, or memory compactness is important. Use a linked list when you need frequent insertions or deletions at the beginning or middle, the size is unknown in advance, or you are implementing stacks and queues. Arrays have better cache performance because elements are stored contiguously, while linked lists have pointer overhead.

Stacks, Queues & Deques

Stack

A stack is a Last-In First-Out (LIFO) data structure. The last element added is the first to be removed. The three core operations are push, which adds an element to the top; pop, which removes the top element; and peek, which views the top element without removing it. All three run in $O(1)$ time. Stacks are used in function call management, expression evaluation, undo functionality, and depth-first search.

Queue

A queue is a First-In First-Out (FIFO) data structure. The first element added is the first to be removed. The enqueue operation adds an element to the rear and dequeue removes from the front, both in $O(1)$ time. Queues are used in OS process scheduling, print spooling, breadth-first search, and any scenario where order of arrival must be preserved.

Deque

A deque (double-ended queue) supports insertion and deletion at both the front and rear in $O(1)$ time. It is a generalization of both stacks and queues. Deques are useful for sliding window problems, palindrome checking, and task scheduling where tasks can be added or removed from either end.

Priority Queue

A priority queue is an abstract data type where each element has a priority. The element with the highest priority is dequeued first, regardless of arrival order. It is typically implemented using a heap, which allows insertion in $O(\log n)$ and extracting the max or min in $O(\log n)$. Priority queues are used in Dijkstra's shortest path algorithm, Huffman coding, and OS job scheduling.

Trees & Graphs

Binary Tree

A tree is a hierarchical data structure with a root node at the top and children below it. A binary tree allows each node to have at most two children, called the left and right child. Key terms include the root, which is the topmost node; a leaf, which is a node with no children; and height, which is the longest path from root to a leaf. Tree traversals include in-order, pre-order, post-order, and level-order (BFS).

Binary Search Tree

A Binary Search Tree (BST) maintains the property that all values in the left subtree are less than the root and all values in the right subtree are greater. This allows search, insertion, and deletion in $O(\log n)$ on average. In the worst case with a skewed tree, these degrade to $O(n)$. Self-balancing BSTs like AVL trees and Red-Black trees maintain balance automatically to guarantee $O(\log n)$ operations.

Heaps

A heap is a complete binary tree satisfying the heap property. In a max-heap, every parent is greater than or equal to its children. In a min-heap, every parent is less than or equal to its children. Heaps are efficiently stored in arrays. Insertion and deletion both run in $O(\log n)$. The heap is the backbone of the priority queue and heap sort algorithm.

Graphs

A graph G consists of vertices V and edges E connecting them. Graphs can be directed, where edges have a direction, or undirected. They can be weighted, where edges have costs, or unweighted. Graphs are represented as adjacency matrices for dense graphs or adjacency lists for sparse ones. BFS explores nodes level by level and finds shortest paths in unweighted graphs. DFS explores as deep as possible and is used for cycle detection and topological sorting. Dijkstra's algorithm finds shortest paths in weighted graphs.